

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

Predmet: Objektovo orientované programovanie

Bohdan Hahalyuk

Forest Knight

Finalne odovzdanie

Bratislava 2026

Obsah

Zámer projektu.....	2
Herná mechanika.....	3
Postavy.....	4
OOP princípy.....	5
Dedenie (Inheritance).....	5
Polymorfizmus (Polymorphism).....	6
Abstrakcia (Abstraction).....	6
Zapuzdrenie (Encapsulation).....	6
Pret'aženie (Overloading).....	6
Prekonanie (Overriding).....	7
Kompozícia (Composition).....	7
Funkcionality a princípy.....	7
GUI.....	8
Testy.....	9
UML diagram tried.....	12
Používateľský manuál.....	13

Zámer projektu

- Hra je jednoduchá survival aréna zasadená do temného lesa po rozpade kráľovstva.
- Hráč ovláda rytiera, ktorý sa snaží prežiť útoky nepriateľov a získať čo najviac bodov.
-
- Počas hry sa hráč pohybuje po mape, bojuje pomocou meča na blízko alebo kuše na diaľku a zbiera šípy z porazených nepriateľov.
- Nepriatelia (vlci, medvede a nepriateľskí rytieri) sa pohybujú smerom k hráčovi a pri kontakte spôsobujú poškodenie.
-
- Cieľom hry je prežiť čo najdlhšie a dosiahnuť požadovaný počet bodov v závislosti od obtiažnosti.

Herná mechanika

- Hráč sa pohybuje pomocou kláves WASD v prostredí typu top-down.
-
- Útok je realizovaný dvoma spôsobmi:
 - - mečom na blízko pomocou hitboxu,
 - - kušou na diaľku pomocou projektilov (šípov).
-
- Počet šípov je obmedzený a je možné ich získať z porazených nepriateľov.
-
- Nepriatelia sa automaticky pohybujú smerom k hráčovi a pri kontakte mu spôsobujú poškodenie.
-
- Za každého porazeného nepriateľa hráč získava body, ktoré slúžia ako hlavný cieľ hry.
-

Postavy

Hlavnou postavou je rytier (Player), ktorý sa pohybuje po mape a bojuje proti nepriateľom.

V hre sa nachádza viacero typov nepriateľov:

- vlci (rýchli, slabší),
- medvede (pomalší, silnejší),
- nepriateľskí rytieri (vyvážené vlastnosti).

OOP princípy

V projekte boli použité základné princípy objektovo orientovaného programovania.

Dedenie (Inheritance)

Dedenie je využité na vytvorenie hierarchie tried:

Enemy (abstraktná trieda):

- EnemyWolf
- EnemyBear
- EnemyKnight

Weapon (abstraktná trieda):

- WeaponSword
- WeaponCrossbow

PowerUp (abstraktná trieda):

- PowerHealth
- PowerDamageIncrease
- PowerSpeedIncrease
- PowerSword

Polymorfizmus (Polymorphism)

Polymorfizmus je využitý pri práci s abstraktnými triedami:

- Weapon.attack() → rôzne správanie:
 - WeaponSword (útok na blízko)
 - WeaponCrossbow (streľba šípov)
- PowerUp.apply() → rôzne efekty:
 - heal (obnovenie HP)
 - zvýšenie damage
 - zvýšenie rýchlosti

Abstrakcia (Abstraction)

Abstrakcia je realizovaná pomocou abstraktných tried:

- Weapon → definuje spoločnú metódu attack()
- PowerUp → definuje metódu apply()

Konkrétna implementácia je ponechaná na odvodené triedy.

Zapuzdrenie (Encapsulation)

Zapuzdrenie je využité v triedach ako Player a Enemy:

- atribúty sú označené ako private (napr. hp, speed, damage)
- prístup je zabezpečený cez metódy (getter, logika triedy)

Pret'aženie (Overloading)

Pret'aženie metód je využité v triede Enemy:

- takeDamage(int damage)
- takeDamage(int damage, boolean isCritical)

Prekonanie (Overriding)

Prekonanie metód je využité pomocou @Override:

- WeaponSword.attack()
- WeaponCrossbow.attack()
- PowerUp.apply()

Kompozícia (Composition)

Kompozícia je využitá v triede GameContext:

- obsahuje Player
- obsahuje zoznam Enemy
- obsahuje zoznam Arrow

Triedy medzi sebou komunikujú prostredníctvom tohto kontextu.

Funkcionalita a princípy

V projekte boli implementované viaceré dodatočné funkcionality a princípy.

Návrhový vzor Strategy

V projekte je použitý návrhový vzor Strategy pri implementácii zbraní.

Abstraktná trieda Weapon definuje spoločnú metódu attack(), pričom konkrétne implementácie:

- WeaponSword
- WeaponCrossbow

majú odlišné správanie. Hráč môže počas hry meniť zbraň, čím sa dynamicky mení spôsob útoku.

PowerUp systém

Podobný princíp je použitý aj pri triede PowerUp, kde rôzne typy power-upov implementujú metódu apply() a poskytujú rôzne efekty hráčovi (liečenie, zvýšenie damage, rýchlosti).

Lambda výrazy

V projekte sú použité lambda výrazy, napríklad pri implementácii hernej slučky pomocou Timer alebo pri práci so zoznamami (napr. removelf).

```
public void updateArrows() {  
    arrows.removeIf(arrow -> arrow.update(context));  
}
```

GameContext

Trieda GameContext slúži ako spoločný objekt na zdieľanie dát medzi rôznymi časťami hry (Player, Enemy, Arrow), čím zjednodušuje komunikáciu medzi objektmi.

GUI

Grafické používateľské rozhranie je implementované pomocou knižnice Swing.

Hlavnou triedou pre vykresľovanie a hernú logiku je GamePanel, ktorý zabezpečuje:

- hlavný herný loop pomocou Timer
- pravidelný update herného stavu
- vykresľovanie všetkých objektov hry

Hra má jednu hlavnú obrazovku (game screen), kde sa zobrazuje herný svet v top-down pohľade.

V rámci GUI sa vykresľujú:

- hráč (Player)
- nepriatelia (Enemy)
- projektily (Arrow)
- power-upy
- používateľské rozhranie (UI)

Grafika je oddelená od logiky hry, pričom renderovanie prebieha samostatne cez metódu render().

Testy

playerShouldShootAndReduceArrows

Overuje, či hráč môže strieľať a či sa správne znižuje počet šípov po výstrele.

attackCooldownShouldDecrease

Overuje, či sa cooldown útoku hráča správne znižuje po aktualizácii hry.

playerShouldMoveUp

Overuje, či sa hráč pohne smerom nahor pri stlačení klávesy W.

playerShouldLoseHP

Overuje, či hráč správne stráca životy po prijatí poškodenia.

swordShouldDamageEnemy

Overuje, či meč spôsobí poškodenie nepriateľovi pri zásahu.

crossbowShouldCreateArrow

Overuje, či kuša vytvorí strelu (Arrow) po útoku.

arrowShouldHitEnemy

Overuje, či šíp správne zasiahne nepriateľa pri kolízii.

enemyShouldTakeDamage

Overuje, či nepriateľ správne stráca HP po zásahu.

enemyShouldTakeCriticalDamage

Overuje, či kritický zásah spôsobí dvojnásobné poškodenie.

enemyShouldBeKnockedBack

Overuje, či nepriateľ dostane knockback efekt po zásahu.

enemyShouldDieAndGiveScore

Overuje, či nepriateľ zomrie po veľkom poškodení a spustí logiku smrti.

arrowShouldNotHitWhenFar

Overuje, či šíp nezasiahne nepriateľa mimo dosahu.

powerUpShouldBeCollected

Overuje, či power-up správne ovplyvní hráča po kolízii.

enemyShouldReceiveKnockback

Overuje, či nepriateľ dostane fyzikálny knockback efekt pri zásahu.

gameWorldShouldUpdateEnemies

Overuje, či herný svet správne aktualizuje nepriateľov.

keyPressedShouldSetMovementFlags

Overuje, či klávesnica správne nastaví pohybové flagy.

keyReleasedShouldUnsetMovementFlags

Overuje, či sa pohybové flagy správne vypnú po pustení kláves.

weaponKeysShouldBeSet

Overuje, či sa správne nastavujú klávesy pre výber zbrane.

gamePanelShouldCreateWithoutCrash

Overuje, či sa GamePanel vytvorí bez chyby.

powerHealthShouldHealPlayer

Overuje, či power-up správne lieči hráča.

powerDamageShouldStackCorrectly

Overuje, či sa bonus na damage správne sčíta.

mousePressedShouldSetClickedTrue

Overuje, či kliknutie myšou nastaví stav inputu.

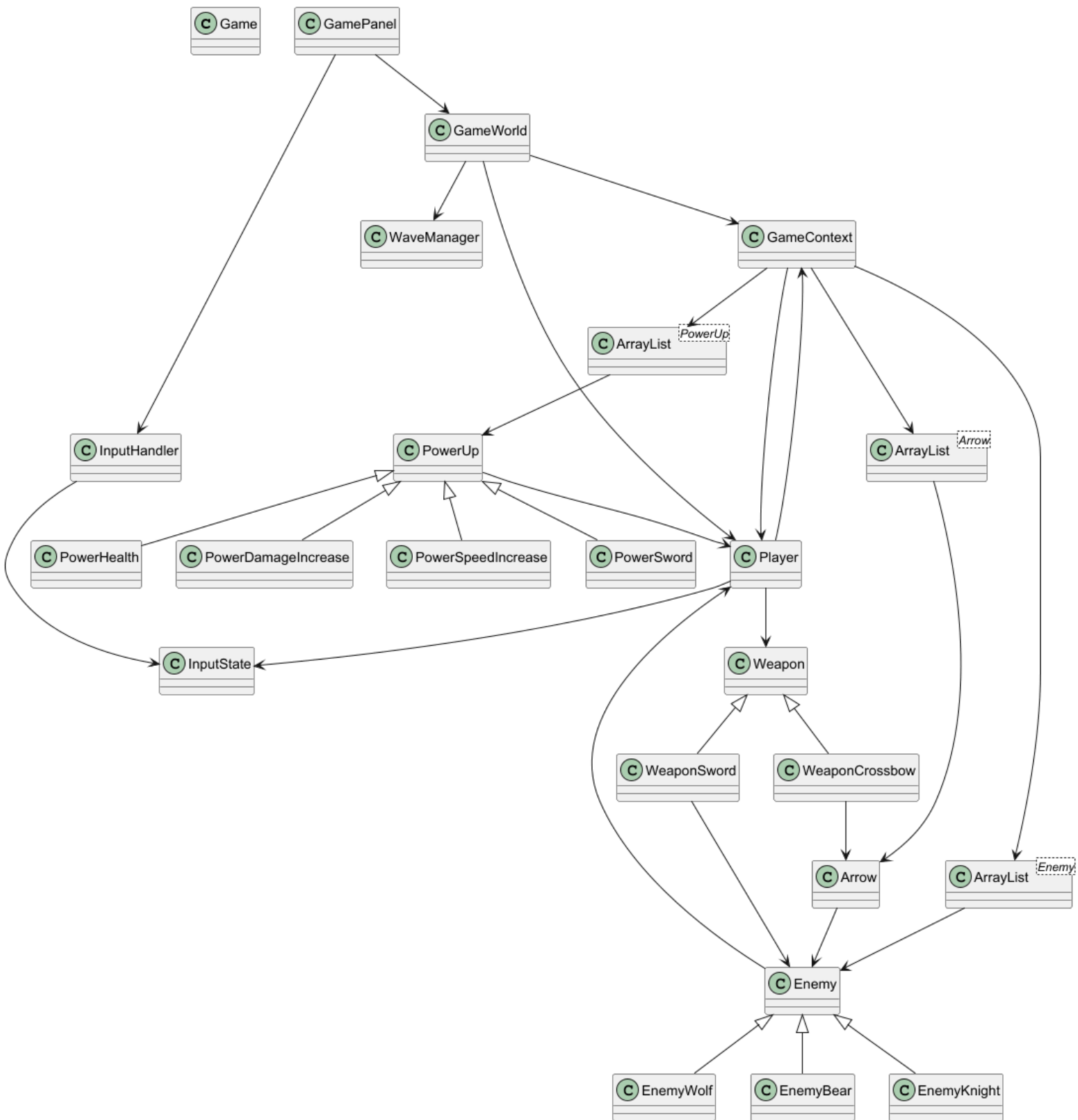
mouseMovedShouldUpdateCoordinates

Overuje, či sa správne aktualizujú súradnice myši.

emptyMouseMethodsShouldRunWithoutError

Overuje, či prázdne metódy event handlerov nespôsobujú chyby.

UML diagram tried



Používateľský manuál

Spustenie programu:

Program sa spúšťa cez triedu `Main`, ktorá inicializuje hru pomocou metódy `Game.start()`.

Ovládanie:

- Pohyb hráča: klávesy W, A, S, D
- Útok: ľavé tlačidlo myši (LKM)
- Ovládanie meča: V závislosti od smeru myši

Prepínanie zbraní:

- kláves 1 → meč (`WeaponSword`)
- kláves 2 → kuša (`WeaponCrossbow`)

Cieľ hry:

Cieľom hry je prežiť čo najdlhšie a získať čo najviac bodov porážaním nepriateľov.

Herné prvky:

- Nepriatelia sa objavujú vo vlnách
- PowerUp objekty zlepšujú schopnosti hráča (napr. damage, speed, health, sword)

PowerUp objekty:

- zlepšujú schopnosti hráča:
- zvýšenie HP
- zvýšenie damage
- zvýšenie rýchlosti
- zvýšenie dĺžky meča (hitbox)

System šípov:

- hráč má obmedzený počet šípov
- šípy sa dajú získať porážaním nepriateľov

Cooldown systém:

- útoky majú cooldown, aby sa zabránilo spamovaniu útokov

Commity

Počas celého semestra bolo v projekte vytvorených 20 commitov, každý týždeň som urobil aspoň dva commity.

1. commit – Initial commit: Forest Knight project
Založenie projektu a základná štruktúra hry.
2. commit – pridaný objekt nepriateľa s jednoduchou AI
Implementácia základného správania nepriateľov.
3. commit – odstránenie hard-codingu
Refaktoring kódu a zlepšenie architektúry.
4. commit – pridaná logika smrti hráča
Hráč môže zomrieť a ukončiť hru.
5. commit – pridaný WaveManager a nové typy nepriateľov
Implementácia systému vln a rozšírenie nepriateľov.
6. commit – pridaný systém meča (sword logic)
Základný melee útok hráča.
7. commit – pridanie UI prvkov (HP, score, wave) a vlnová logika
Zobrazenie základných herných informácií.
8. commit – pridané power-upy
Implementácia zberateľných vylepšení hráča.
9. commit – rozšírenie hernej logiky, viac vln a power-upov
Zvýšenie komplexity hry.
10. commit – vizuálny cooldown meča
Pridanie vizuálnej spätnej väzby pri útoku.
11. commit – knockback systém a vizuálny efekt zranenia nepriateľov
Nepriatelia reagujú na zásahy spätným odrazom a efektom poškodenia.
12. commit – abstraktná trieda Weapon a refaktoring zbraní
Zavedenie Strategy pattern pre zbrane.
13. commit – implementácia streľby šípov a nového typu zbrane
Pridanie ranged útoku (kuša a šípy).
14. commit – pridanie grafických assetov objektov
Vylepšenie vizuálnej stránky hry.
15. commit – pridanie pozadia a obrázkov
Implementácia herného pozadia.

16. commit – checkpoint refaktoring logiky

Úprava a stabilizácia hernej logiky.

17. commit – reorganizácia súborov a zlepšenie OOP štruktúry

Zaradenie tried do balíkov a odstránenie porušenia OOP princípov.

18. commit – pridanie rozhraní (interfaces) a finalizácia projektu

Zavedenie interface a príprava projektu na dokončenie.

19. commit – projekt je pripravený na odovzdanie.

20. commit – pridaná tretia záverečná dokumentácia.

.